

for communication between the OS and I/O devices, two alternatives:

- special machine language instructions
- **memory-mapped I/O**: dedicate a portion of the physical address space to *registers within the I/O device controllers*, and access using **ordinary load and store instructions**

example of memory-mapped I/O: reading a character from a simple keyboard device with a control/status register and a data register, using polling in a “busy-wait” loop

**control/status register:** includes a “ready bit” that is set to 1 *by device controller* when new character typed (assume here all other bits read as 0)

**data register:** when new character typed, put in this register *by device controller*

[illegible]

## hard disk drives (magnetic disks)

consist of 1-4 platters, each side of which can be used for recording data (by magnetizing spots on surface)

disk “arm” has one **read/write head per surface**

each surface divided into **tracks**, and each track is divided into **sectors** (4 KiB sector size)

three major components of disk access time

- **seek time**: time to **move arm** to the required track (e.g., could be > 20 ms for a “long” seek, with average 3-4 ms)
- **rotational latency**: time until first sector of data to be read/written rotates under the read/write head (e.g., for a 10,000 RPM disk, what would average rotational latency be?)
- **transfer time**: time from when the first bit is read/written until last bit is read/written (e.g., for 200 MB/sec transfer *rate*, what is time for 4 KB?)

operating systems typically will **read/write multiple sectors at once if possible** (... *why?*)

**caching** is used, both within the disk drive itself, and in main memory

## SSDs (“solid-state drives”)

use integrated circuit technology for data storage, and so **no moving parts**, but like magnetic disk data is retained when powered off

commonly, based on **NAND flash memory** (different types: **SLC**, **MLC**, **TLC**, **QLC**)

flash chips organized into **banks** (or “planes”), each with many **erase blocks** (e.g., each 16 MiB), each of which consists of many **pages** (e.g., each 16 KiB)

before writing to (“programming”) an individual page, *entire block must be erased* (takes a few ms!)

after many repeated program/erase cycles, **storage cells wear out** and become unusable

how to get an **SSD** from a collection of flash chips?

- can use same **block-based interface** to rest of system as a hard disk drive
- **flash translation layer** handles mapping to bank/erase block/page, erase operations, “wear leveling”, ...

**cost**, **speed**, **typical capacity** intermediate between disk & main memory, **power usage** lower than both

*historically, dramatic reductions in storage costs!*